

AWS WordPress Hosting Using VPC, Public and Private Subnets

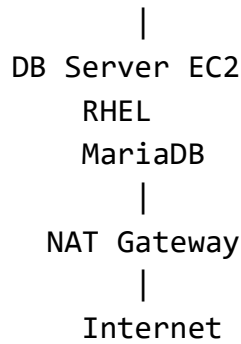
1. Description

- i. Launch two AWS EC2 instances, one in a public subnet and one in a private subnet.
- ii. Host a **WordPress website** on the public web server.
- iii. Host the **database** on the private database server.
- iv. Configure the database server to accept connections only from its web server and prevent public access to the database server, even if the security group has open entries.
- v. Provide internet access to the private database server through a **NAT Gateway** to perform software package updates, patching, and other required updates.

2. AWS Architecture

The following architecture was implemented:





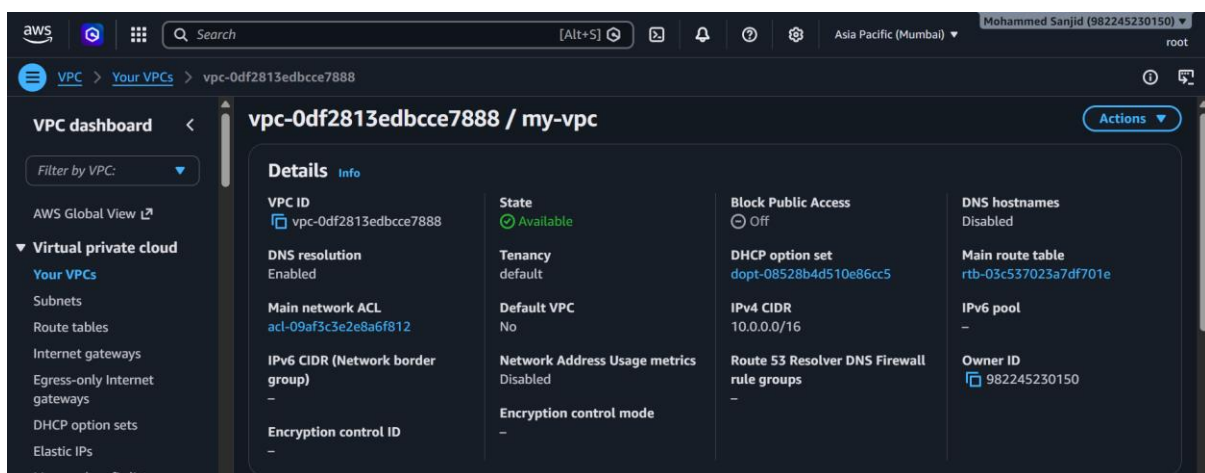
The Web Server was placed in the public subnet so that WordPress could be accessed through its public IP address. The Database Server was placed in the private subnet without a public IPv4 address.

3. VPC Configuration

A VPC named **myvpc** was created for the WordPress hosting environment.

Configuration	Value
VPC Name	myvpc
IPv4 CIDR	10.0.0.0/16
IPv6	Not configured
Tenancy	Default

The VPC provided the isolated network environment for the Web Server and Database Server.

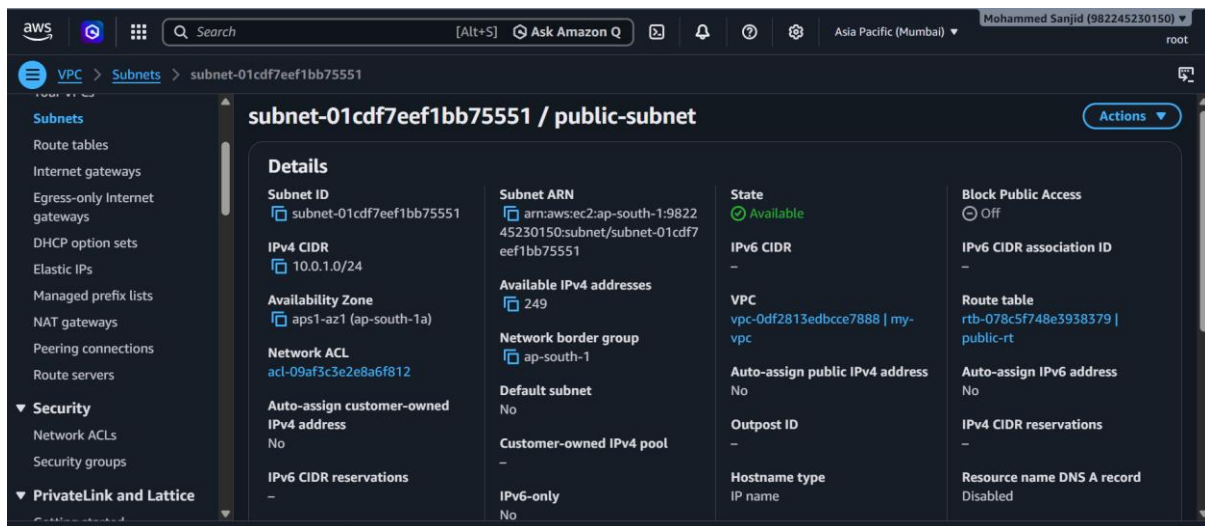


4. Public Subnet

A public subnet named **public-subnet** was created inside myvpc.

Configuration	Value
Subnet Name	public-subnet
VPC	myvpc
CIDR	10.0.1.0/24
Availability Zone	Selected lab Availability Zone

The public subnet was used for the WordPress Web Server.

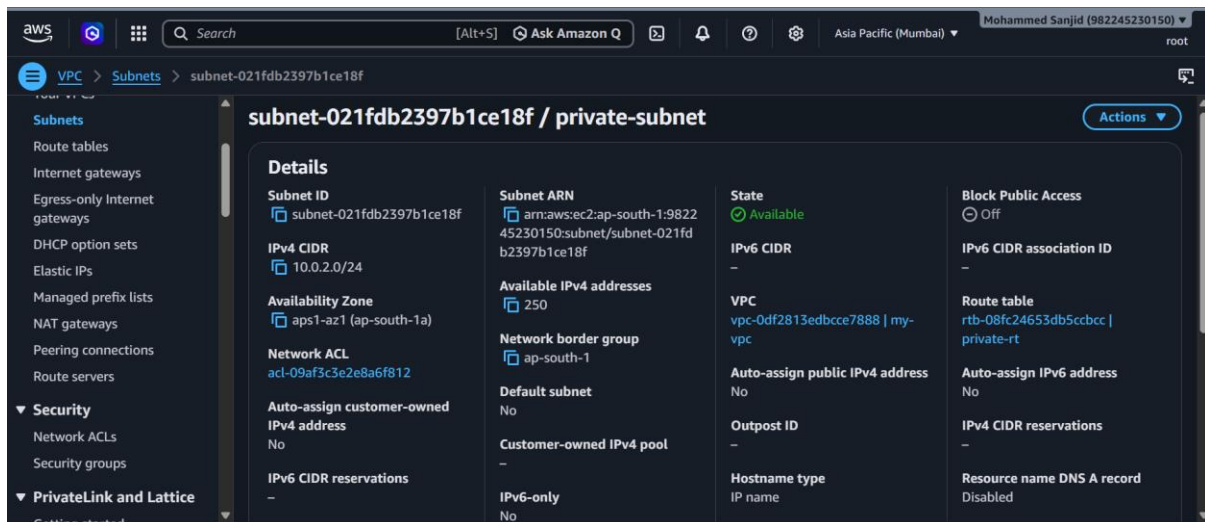


5. Private Subnet

A private subnet named **private-subnet** was created inside myvpc.

Configuration	Value
Subnet Name	private-subnet
VPC	myvpc
CIDR	10.0.2.0/24
Availability Zone	Selected lab Availability Zone

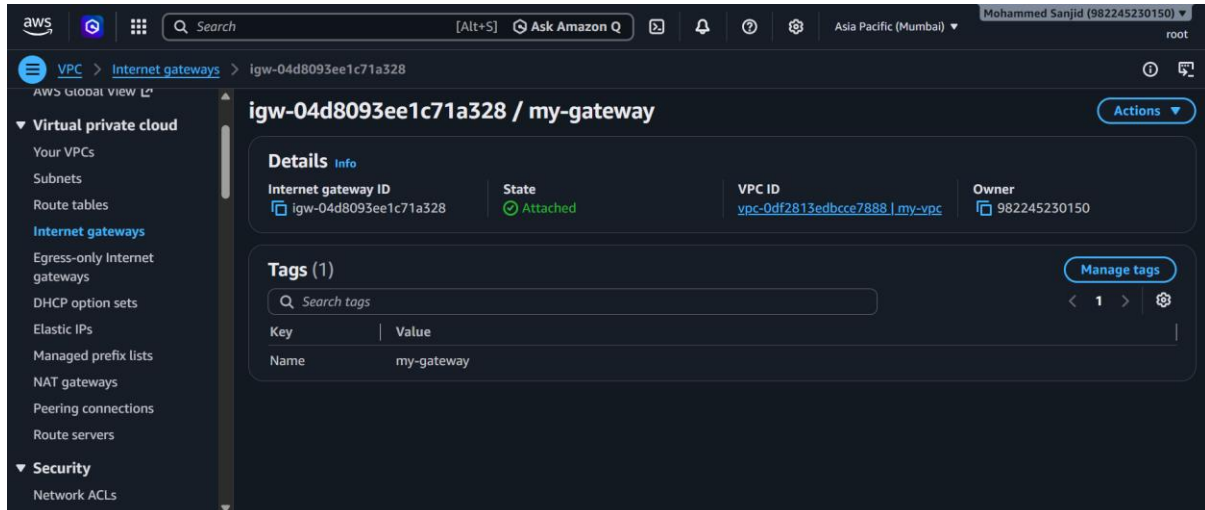
The private subnet was used for the Database Server.



6. Internet Gateway

A gateway named **my-gateway** was created and attached to the VPC myvpc.

The gateway provided Internet connectivity for resources in the public subnet.



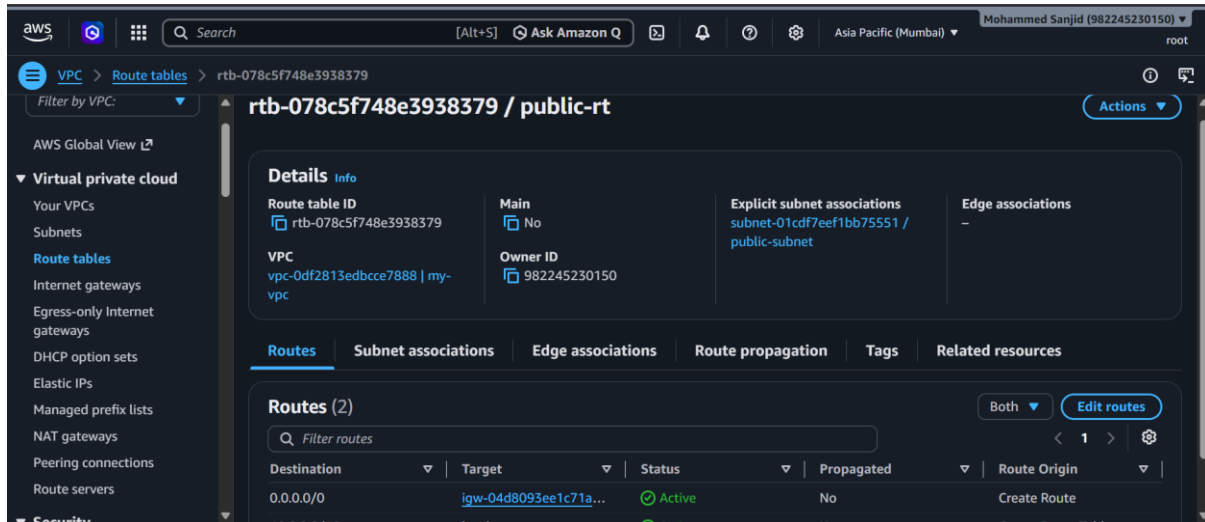
7. Public Route Table

A route table named **public-rt** was created and associated with public-subnet.

The following route was configured:

Destination	Target
0.0.0.0/0	my-gateway

This route allowed the Web Server in the public subnet to communicate with the Internet.

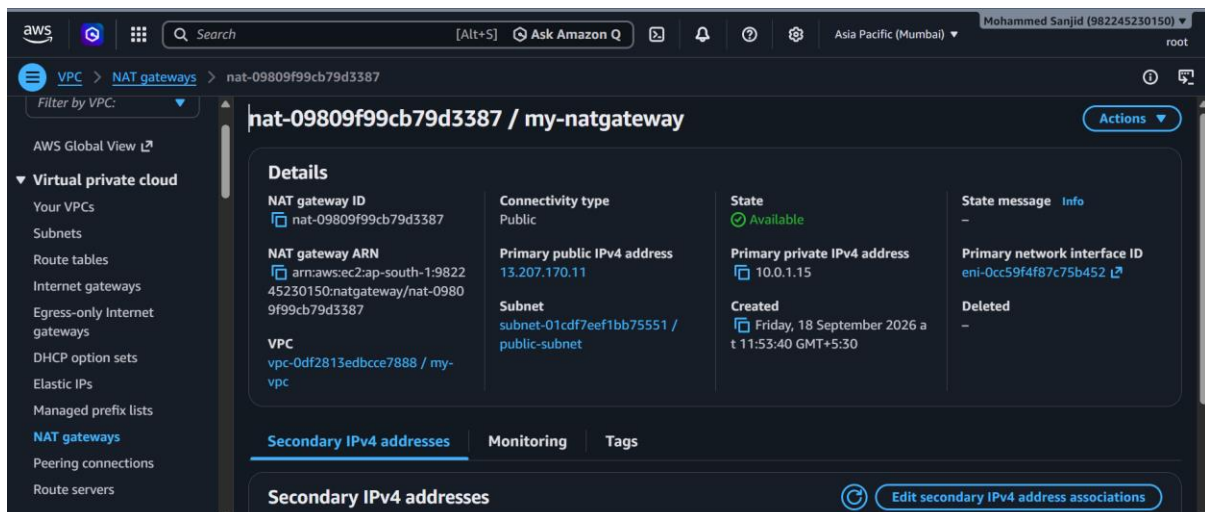


8. NAT Gateway

A **public NAT Gateway** was created inside public - subnet.

An Elastic IP address was allocated to the NAT Gateway.

The NAT Gateway provided outbound Internet connectivity to resources in private - subnet without assigning a public IP address to the Database Server.



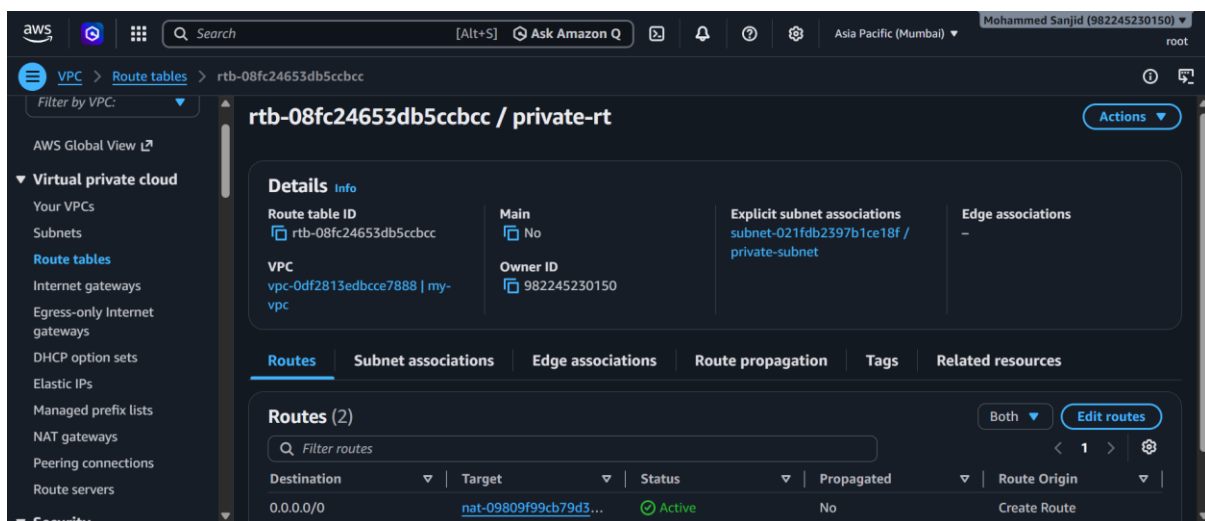
9. Private Route Table

A route table named **private-rt** was created and associated with private-subnet.

The following route was configured:

Destination	Target
0.0.0.0/0	NAT Gateway

This allowed the Database Server in the private subnet to access the Internet for outbound operations through the NAT Gateway.



10. Security Groups

10.1 Web Server Security Group

A security group named **webserver-sg** was created for the Web Server.

Inbound Rules

Type	Port	Source
SSH	22	My IP
HTTP	80	0.0.0.0/0

The Web Server was therefore accessible through SSH from the configured IP and through HTTP from the Internet.

Outbound Rules

All traffic was allowed.

10.2 Database Server Security Group

A security group named **dbserver-sg** was created for the Database Server.

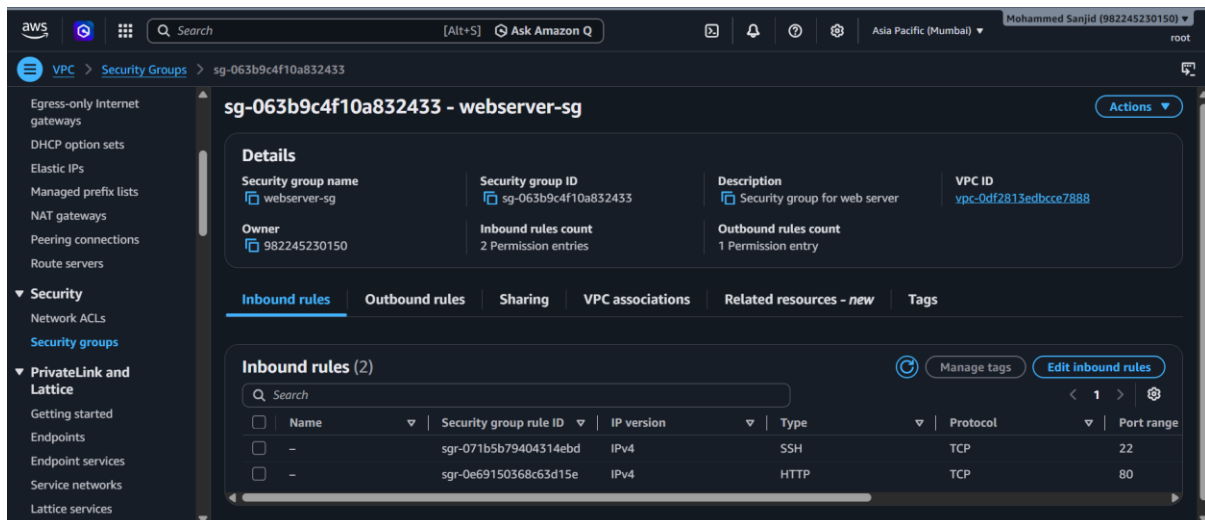
Inbound Rules

Type	Port	Source
MySQL/Aurora	3306	webserver-sg
SSH	22	webserver-sg

Outbound Rules

All traffic was allowed.

The Database Server was not directly exposed to the Internet. Port 3306 was allowed only from webserver-sg.

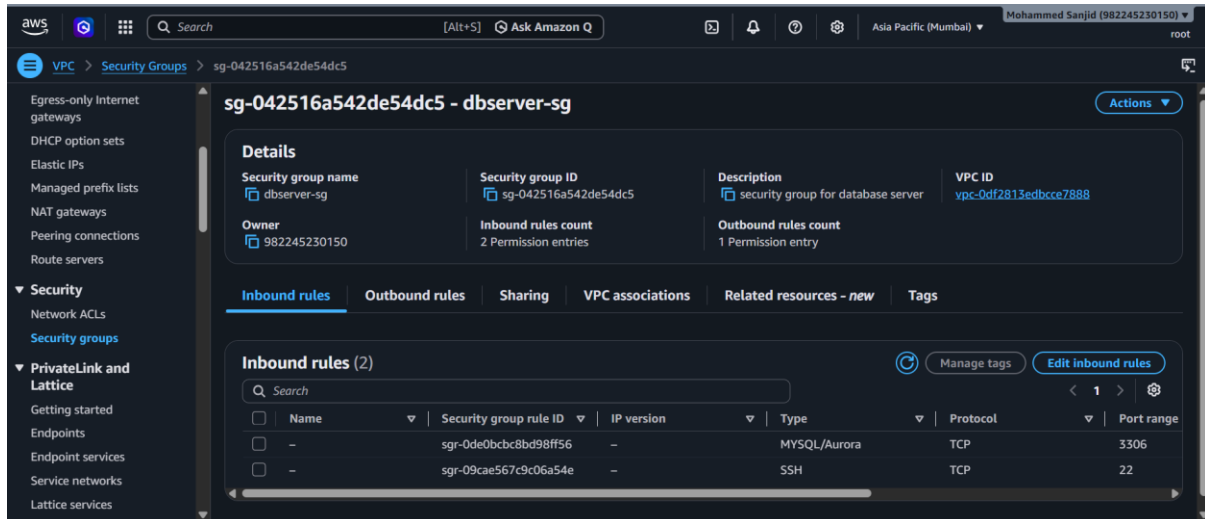


The screenshot shows the AWS Management Console interface for the security group **sg-063b9c4f10a832433 - webserver-sg**. The console displays the following details:

- Security group name:** webserver-sg
- Security group ID:** sg-063b9c4f10a832433
- Description:** Security group for web server
- VPC ID:** vpc-0df2813edbcce7888
- Owner:** 982245230150
- Inbound rules count:** 2 Permission entries
- Outbound rules count:** 1 Permission entry

The **Inbound rules** section shows two rules:

Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sgr-071b5b79404314ebd	IPv4	SSH	TCP	22
-	sgr-0e69150368c63d15e	IPv4	HTTP	TCP	80



11. Web Server EC2 Instance

An EC2 instance named **Web-Server** was launched using a RHEL AMI.

The instance was configured as follows:

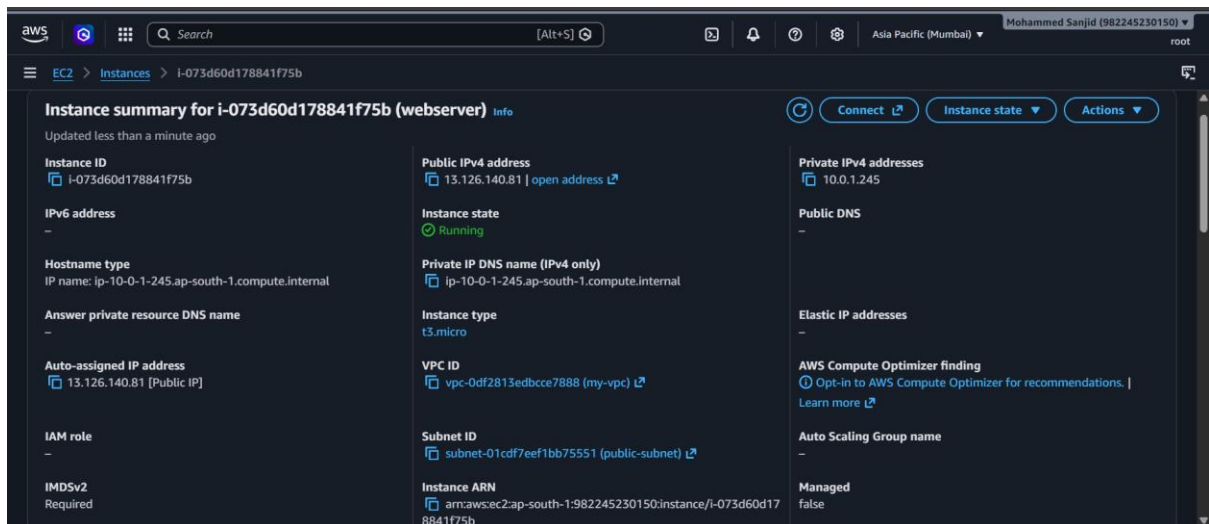
Configuration	Value
Name	Web-Server
Operating System	RHEL
Subnet	public-subnet
Public IPv4	Enabled
Security Group	webserver-sg
Key Pair	Existing lab key pair

The Web Server received:

Private IP: 10.0.1.245

Public IP: 13.126.140.81

The Web Server was used to host Nginx, PHP, PHP-FPM and WordPress.



12. Database Server EC2 Instance

An EC2 instance named **DB-Server** was launched using a RHEL AMI.

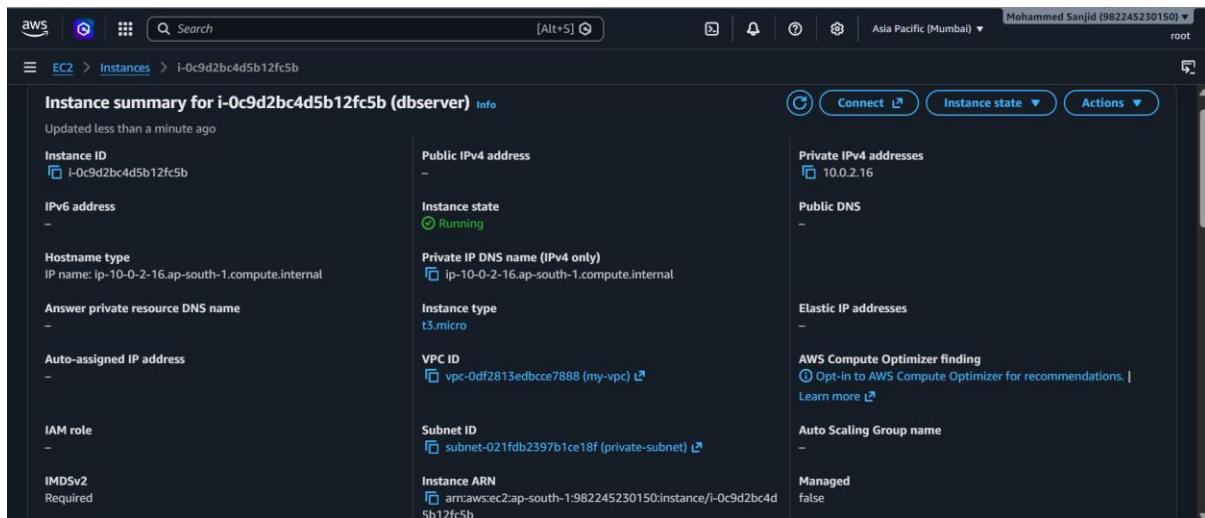
The instance was configured as follows:

Configuration	Value
Name	DB-Server
Operating System	RHEL
Subnet	private-subnet
Public IPv4	Disabled
Security Group	dbserver-sg
Key Pair	Existing lab key pair

The Database Server received the private IP:

10.0.2.16

The Database Server did not have a public IPv4 address.



13. SSH Connection to the Web Server

The Web Server was accessed from Windows using the configured SSH key:

```
ssh -i "C:\Users\mhdsa\Downloads\sanjidnew.pem" ec2-user@13.126.140.81
```

A successful SSH connection to the Web Server was established.

```
mhdsa@HP-DESKTOP MINGW64 ~/Downloads
$ ssh -i "sanjidnew.pem" ec2-user@13.126.140.81
Register this system with Red Hat Lightspeed: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Lightspeed will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
Last login: Fri Sep 18 09:52:04 2026 from 49.47.196.102
[ec2-user@ip-10-0-1-245 ~]$
```

14. SSH Connection to the Database Server

Since the Database Server did not have a public IP address, it was accessed through the Web Server using its private IP address.

From the Web Server:

```
ssh -i ~/sanjidnew.pem ec2-user@10.0.2.16
```

A successful SSH connection from the Web Server to the Database Server was established.

```
[ec2-user@ip-10-0-1-245 ~]$ ssh -i ~/sanjidnew.pem ec2-user@10.0.2.16
Register this system with Red Hat Lightspeed: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Lightspeed will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
Last login: Fri Sep 18 09:06:02 2026 from 10.0.1.245
[ec2-user@ip-10-0-2-16 ~]$
```

15. MariaDB Installation

MariaDB was installed on the Database Server:

```
sudo dnf install mariadb-server -y
```

The MariaDB service was enabled and started:

```
sudo systemctl enable --now mariadb
```

The MariaDB service was verified to be running.

16. MariaDB Remote Connection Configuration

The MariaDB configuration file was edited:

```
sudo vi /etc/my.cnf.d/mariadb-server.cnf
```

The following configuration was added under the [mysqld] section:

```
bind-address=0.0.0.0
```

MariaDB was restarted:

```
sudo systemctl restart mariadb
```

MariaDB was verified to be listening on port 3306:

```
sudo ss -lntp | grep 3306
```

The database server was successfully configured to accept connections through the private network.

17. WordPress Database Creation

The MariaDB console was opened:

```
sudo mysql
```

The WordPress database was created:

```
CREATE DATABASE myappdb;
```

A database user was created:

```
CREATE USER 'webuser'@'%' IDENTIFIED BY 'Web@12345';
```

Privileges were granted:

```
GRANT ALL PRIVILEGES ON myappdb.* TO 'webuser'@'%' ;
```

The privileges were applied:

```
FLUSH PRIVILEGES;
```

The database configuration was:

Configuration	Value
Database	myappdb
User	webuser
Password	Web@12345
Database Server	10.0.2.16
Port	3306

18. Database Connectivity Verification

The MariaDB client was installed on the Web Server:

```
sudo dnf install mariadb -y
```

The database connection was tested using:

```
mysql -h 10.0.2.16 -u webuser -p
```

Connectivity to port 3306 was also verified:

```
nc -zv 10.0.2.16 3306
```

The connection was successfully established:

```
Ncat: Connected to 10.0.2.16:3306.
```

This confirmed that the Web Server could communicate with the Database Server through the private network.

19. Nginx Installation

Nginx was installed on the Web Server:

```
sudo dnf install nginx -y
```

The Nginx service was enabled and started:

```
sudo systemctl enable --now nginx
```

The Nginx service was verified to be running.

20. PHP and PHP-FPM Installation

The required PHP packages were installed:

```
sudo dnf install php php-fpm php-mysqlnd php-gd php-xml php-mbstring  
php-json php-curl php-zip -y
```

PHP-FPM was enabled and started:

```
sudo systemctl enable --now php-fpm
```

PHP-FPM was used to process the PHP files required by WordPress.

21. WordPress Download

WordPress was downloaded on the Web Server.

The /tmp directory was used:

```
cd /tmp
```

WordPress was downloaded:

```
curl -O https://wordpress.org/latest.tar.gz
```

The archive was extracted:

```
tar -xzf latest.tar.gz
```

This created:

```
/tmp/wordpress
```

22. Copy WordPress to the Nginx Web Root

The existing Nginx web root was cleared:

```
sudo rm -rf /usr/share/nginx/html/*
```

The WordPress files were copied:

```
sudo cp -r /tmp/wordpress/* /usr/share/nginx/html/
```

The WordPress files were verified:

```
wp-admin  
wp-content  
wp-includes  
index.php  
wp-config-sample.php
```

The WordPress application was located at:

```
/usr/share/nginx/html/
```

23. WordPress File Permissions

Ownership of the WordPress files was changed to the Nginx user:

```
sudo chown -R nginx:nginx /usr/share/nginx/html/
```

Directory permissions were configured:

```
sudo find /usr/share/nginx/html/ -type d -exec chmod 755 {} \;
```

File permissions were configured:

```
sudo find /usr/share/nginx/html/ -type f -exec chmod 644 {} \;
```

This allowed Nginx to access and serve the WordPress files.

24. SELinux Configuration

The correct SELinux contexts were restored for the WordPress directory:

```
sudo restorecon -Rv /usr/share/nginx/html/
```

The SELinux boolean required for the Web Server to connect to the database was enabled:

```
sudo setsebool -P httpd_can_network_connect_db 1
```

The setting was verified:

```
getsebool httpd_can_network_connect_db
```

The result was:

```
httpd_can_network_connect_db --> on
```

The SELinux context of wp-config.php was also verified:

```
ls -Z /usr/share/nginx/html/wp-config.php
```

The file had the required SELinux type:

```
httpd_sys_content_t
```

25. Nginx Configuration for WordPress

A dedicated Nginx configuration file was created:

```
sudo vi /etc/nginx/conf.d/wordpress.conf
```

The following configuration was used:

```
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ /\.php$ {
        include /etc/nginx/default.d/*.conf;
        fastcgi_pass unix:/run/php-fpm/www.sock;
        fastcgi_param SCRIPT_FILENAME
$document_root$fastcgi_script_name;
        include fastcgi_params;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

The Nginx configuration was tested:

```
sudo nginx -t
```

The configuration test was successful.

Nginx was restarted:

```
sudo systemctl restart nginx
```

26. WordPress Database Configuration

The WordPress configuration file was created from the sample file:

```
cd /usr/share/nginx/html
sudo cp wp-config-sample.php wp-config.php
```

The configuration file was edited:

```
sudo vi wp-config.php
```

The database settings were configured as:

```
define( 'DB_NAME', 'myappdb' );
define( 'DB_USER', 'webuser' );
define( 'DB_PASSWORD', '1234' );
define( 'DB_HOST', '10.0.2.16' );
```

The private IP address 10.0.2.16 was used as the database host, allowing WordPress to communicate with MariaDB through the private network.

27. WordPress Installation

The WordPress website was accessed through the Web Server's public IP:

<http://13.126.140.81>

The WordPress installation was completed successfully.

The administrator dashboard was then accessed through:

<http://13.126.140.81/wp-admin/>

The WordPress Dashboard was successfully displayed.

28. Final Validation

The complete environment was successfully validated.

The following components were confirmed:

- myvpc was created with CIDR 10.0.0.0/16.

- `public`-subnet was configured for the Web Server.
- `private`-subnet was configured for the Database Server.
- `my`-gateway was attached to the VPC.
- `public`-rt routed Internet traffic through `my`-gateway.
- A NAT Gateway provided outbound connectivity for the private subnet.
- `private`-rt routed outbound traffic through the NAT Gateway.
- `webserver`-sg allowed HTTP and controlled SSH access.
- `dbserver`-sg restricted database access to the Web Server.
- The Web Server was deployed in the public subnet.
- The Database Server was deployed in the private subnet.
- The Database Server did not have a public IPv4 address.
- MariaDB was installed and configured.
- The `myappdb` database was created.
- The `webuser` database account was configured.
- Port 3306 connectivity between the Web Server and Database Server was verified.
- Nginx was installed and configured.
- PHP and PHP-FPM were installed.
- WordPress was installed under `/usr/share/nginx/html/`.
- SELinux was configured.
- WordPress was connected to the private MariaDB server.
- The WordPress Dashboard was successfully accessed.

29. Final Output

The WordPress website was successfully hosted on the AWS Web Server.

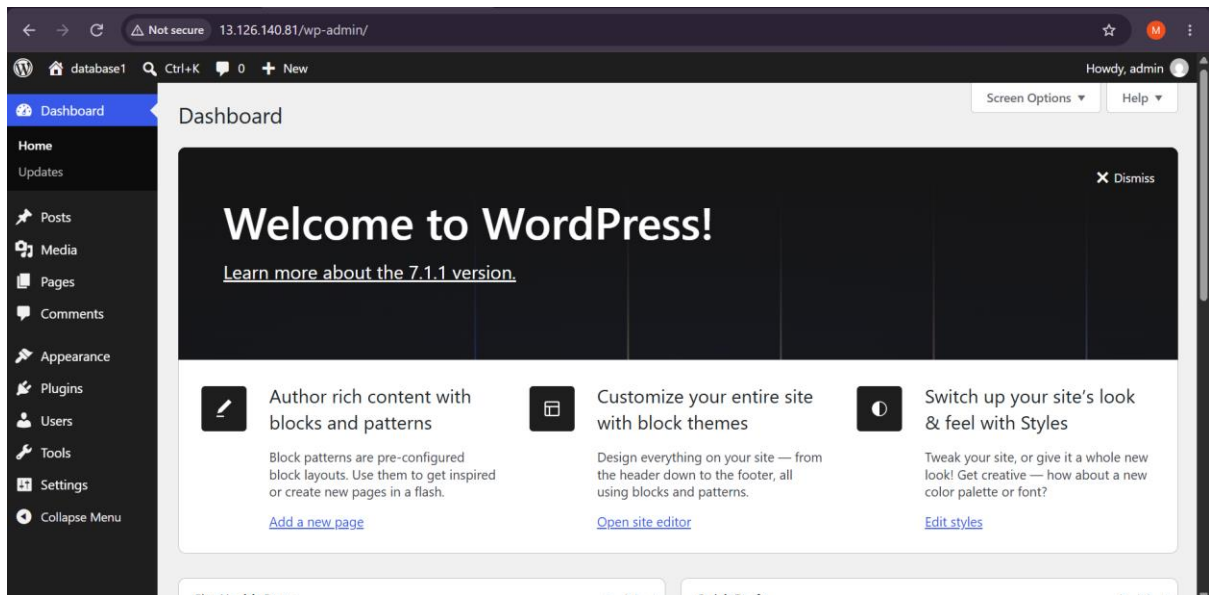
Website URL

<http://13.126.140.81>

WordPress Admin URL

<http://13.126.140.81/wp-admin/>

The WordPress Dashboard was successfully displayed, confirming that the WordPress application and database connection were working correctly.



30. AWS Resource Summary

Resource	Configuration
VPC	myvpc
VPC CIDR	10.0.0.0/16
Public Subnet	public-subnet
Public CIDR	10.0.1.0/24
Private Subnet	private-subnet
Private CIDR	10.0.2.0/24
Gateway	my-gateway
Public Route Table	public-rt
Private Route Table	private-rt
Web Security Group	webserver-sg
DB Security Group	dbserver-sg
Web Server	Web-Server
Web Server Private IP	10.0.1.245
Web Server Public IP	13.126.140.81
Database Server	DB-Server
DB Server Private IP	10.0.2.16
Web Server Software	Nginx, PHP, PHP-FPM, WordPress
Database Software	MariaDB
Database Name	myappdb
Database User	webuser
Database Port	3306

31. Conclusion

The WordPress hosting environment was successfully implemented on AWS using a custom VPC with separate public and private subnets.

The **Web Server** was deployed in `public` - subnet and configured with RHEL, Nginx, PHP, PHP-FPM and WordPress. The **Database Server** was deployed in `private` - subnet and configured with RHEL and MariaDB without a public IPv4 address.

The Web Server successfully communicated with the Database Server through the private IP address `10.0.2.16` over TCP port `3306`. The `dbserver-sg` security group restricted database access to the Web Server, while the NAT Gateway provided outbound Internet connectivity for the private subnet.

SELinux was configured to allow the WordPress application to communicate with the database. The WordPress database configuration was completed successfully, and the **WordPress Administrator Dashboard was successfully accessed through the Web Server's public IP address.**